Il modello di Von Neumann

Prof. Rizzotti

1 Modello di Von Neumann

Il modello di Von Neumann descrive l'architettura fondamentale di un computer, definendo i principali componenti hardware e il modo in cui collaborano per eseguire i programmi.

Gli elementi essenziali sono:

- CPU (Central Processing Unit);
- RAM (Random Access Memory);
- Periferiche di Input/Output (I/O).

1.1 CPU - Central Processing Unit

La CPU è l'unità centrale di elaborazione del computer. È responsabile di tre operazioni fondamentali:

- Recupero delle istruzioni;
- Decodifica delle istruzioni;
- Esecuzione delle istruzioni.

1.2 RAM - Random Access Memory

La RAM è una memoria volatile, cioè il suo contenuto viene perso quando si spegne il computer.

È una memoria con tempi di accesso molto più veloci rispetto alle memorie di massa, come HDD e SSD.

Per questo motivo, viene utilizzata per caricare i programmi e i dati necessari durante la loro esecuzione.

1.3 Periferiche di Input/Output (I/O)

Le periferiche di Input/Output sono necessarie per permettere l'interazione tra l'essere umano e la macchina.

Si dividono in due categorie:

- **Input (ingresso):** dispositivi che permettono all'utente di fornire informazioni al computer.
  - Tastiera;
  - Microfono;
  - Mouse.
- **Output (uscita):** dispositivi che permettono al computer di comunicare informazioni all'utente.
  - Monitor (terminale o schermo);
  - Cuffie;
  - Casse.

2 CPU - Central Processing Unit

La CPU (Central Processing Unit) è il componente hardware che coordina l'esecuzione delle istruzioni dei programmi e l'interazione uomo-macchina tramite le periferiche I/O.

È quindi responsabile dell'esecuzione di tutti i programmi (firmware, BIOS, sistema operativo, browser, Steam...) ed esegue le istruzioni in maniera **sequenziale**.

I componenti principali della CPU sono:

- **ALU (Arithmetic Logic Unit)**

  Responsabile delle operazioni matematiche base (+, -, *, /).

  Si può dimostrare che, in ultima analisi, un programma non è altro che una sequenza di operazioni riconducibili alle quattro operazioni matematiche di base e alle operazioni logiche (AND, OR).

  Questo è tutto ciò che è in grado di fare la ALU, e tutto ciò che fa un programma in termini di elaborazione dei dati.

- **Registri**

  Piccole memorie dove salvare gli operandi e il risultato dell'operazione corrente.

- **CU (Control Unit)**

  Responsabile della modifica dei registri, fra cui il Program Counter (PC), ovvero un registro particolare che indica quale istruzione dovrà essere eseguita nel prossimo ciclo di clock.

  La CU gestisce inoltre i registri contenenti eventuali operandi dell'operazione corrente e il risultato della ALU.

- **Cache (L1, L2, L3)**

  Piccole memorie, che adesso arrivano anche ai 64 MB nel livello più inferiore, utilizzate per la temporanea memorizzazione delle istruzioni che i programmi in esecuzione riutilizzano più frequentemente.

  Sono necessarie per ridurre i tempi di esecuzione dei programmi, dato che il recupero delle istruzioni direttamente dalla memoria RAM è dispendioso in termini di tempo.

2.1 Il ciclo macchina (Machine Cycle)

Il ciclo macchina (chiamato anche *Clock Cycle* o ciclo di clock) è l'algoritmo che descrive il funzionamento della CPU ed è costituito da tre fasi:

1. **Recupero (*Instruction Fetch*):** recupero di un'istruzione dalla memoria del programma.
2. **Decodifica (*Decode*):** interpretazione dell'istruzione, ovvero identificazione dell'operazione da eseguire. Si tratta di un'addizione? Una sottrazione? Oppure di un'istruzione di salto?
3. **Esecuzione (*Execute*):** esecuzione dell'istruzione e memorizzazione dell'eventuale risultato.

La frequenza base dei processori attuali può raggiungere addirittura i 5 GHz, equivalenti a cinque miliardi di cicli di clock al secondo.

È proprio grazie a questa velocità che anche una CPU con un solo core (dove per *core* si intende un'unità di esecuzione indipendente, composta da ALU, CU e registri) può darci l'illusione di eseguire più programmi contemporaneamente.

In realtà, la CPU viene utilizzata dai vari programmi in maniera sequenziale, assegnando a ciascuno un piccolo intervallo di tempo, ad esempio appena 10 cicli di clock, prima di passare all'esecuzione di un altro programma.

Considerando una frequenza di 5 GHz, abbiamo a disposizione cinque miliardi di cicli di clock al secondo: ciò significa che ogni ciclo dura appena 0,2 nanosecondi. Di conseguenza, se la CPU dedica anche 1000 cicli di clock a ciascun programma, il cambio di programma avviene ogni 200 nanosecondi, un intervallo di tempo assolutamente impercettibile per noi esseri umani!

2.2 Programmi, processi, thread e CPU multi-core

Un programma (prendiamo il nostro browser Firefox come esempio) è costituito da più parti. Sì, all'essenziale è un insieme di istruzioni macchina, ma queste istruzioni possono essere raggruppate in più funzionalità indipendenti fra loro.

Un browser, per esempio, può avere più pagine aperte, e il nostro computer consente l'apertura e l'esecuzione di più pagine (SoundCloud in una pagina, YouTube in un'altra e Classroom in un'altra ancora).

L'esecuzione del programma prende il nome di **processo**. Un processo può quindi essere suddiviso in più parti, chiamate **thread**, indipendenti o meno fra loro.

Nel caso del nostro browser Firefox, possiamo immaginare che ogni pagina aperta corrisponda a un thread, ovvero a una porzione di lavoro del programma. Ad esempio, mentre ascoltiamo musica su SoundCloud, guardiamo un video su YouTube e lavoriamo su Classroom, Firefox deve gestire contemporaneamente le attività associate a ciascuna pagina.

Una CPU dotata di più core può distribuire questi thread su diverse unità di elaborazione, permettendo di eseguirli realmente in parallelo, anziché alternarli nel tempo come avviene su una CPU con un solo core.

Un **core** è infatti un'unità di elaborazione indipendente, dotata della propria ALU, CU e dei propri registri.

Di conseguenza, su una CPU multi-core è possibile eseguire contemporaneamente thread appartenenti allo stesso programma (ad esempio, diverse pagine di Firefox) oppure a programmi completamente diversi (ad esempio, Firefox e Steam). Questo permette di sfruttare più unità di elaborazione in parallelo, aumentando la quantità di lavoro che la CPU può svolgere nello stesso intervallo di tempo.
[1]

[1] Parti di questo documento sono output di LLM.